Identifier	UC-01
Name	Register Account
Summary	A new user registers an account on the PocketPlate system to access its features.
Priority	High
Actors	User
Pre-condition(s)	User does not have an existing account. User has access to the application.
Post-condition(s)	A new user account is created and the user is redirected to the login page.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User navigates to the registration page and clicks Sign Up.	
2.		System displays the registration form with required fields.
3.	User enters name, email, password, and other required information.	
4.	User clicks the Create Account button.	
5.		System validates the entered details.
6.		System creates the account and sends a confirmation email.
7.		System redirects the user to the login page.

Alternate Course of Action		
S#	**Actor Action**	**System Response**
6.		System displays an error indicating invalid or duplicate email.
7.	User corrects the details and resubmits.	
8.		(5) Continue from S# 5.

Identifier	UC-02
Name	Login
Summary	A registered user or admin logs into the PocketPlate system using their credentials.
Priority	High
Actors	User, Admin
Pre-condition(s)	The actor has a registered account. The application is accessible.
Post-condition(s)	The actor is authenticated and redirected to the home/dashboard page.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	Actor navigates to the login page.	
2.		System displays the login form.
3.	Actor enters email and password.	
4.	Actor clicks the Login button.	
5.		System validates credentials.
6.		System authenticates the actor and redirects to the dashboard.

Alternate Course of Action		
S#	**Actor Action**	**System Response**
6.		System displays an error: Invalid email or password.
7.	Actor re-enters credentials.	
8.		(5) Continue from S# 5.

Identifier	UC-03	
Name	Manage Profile	
Summary	A logged-in user updates their personal profile information such as name, photo, or contact details.	
Priority	Medium	
Actors	User	
Pre-condition(s)	User is logged in. User has an existing profile.	
Post-condition(s)	User profile information is updated and saved in the system.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User navigates to the Profile section.	
2.		System displays current profile details.
3.	User edits the desired fields (name, photo, contact info, etc.).	
4.	User clicks Save Changes.	
5.		System validates the updated information.
6.		System saves changes and displays a success confirmation.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
6.		System highlights invalid fields with error messages.
7.	User corrects the fields.	
8.		(5) Continue from S# 5.

Identifier	UC-04	
Name	Set Preferences	
Summary	User sets food preferences such as dietary restrictions, cuisine preferences, and notification settings.	
Priority	Medium	
Actors	User	
Pre-condition(s)	User is logged in.	
Post-condition(s)	User preferences are saved and used for personalised recommendations.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User navigates to the Preferences section.	
2.		System displays preference options (dietary, cuisine, notifications).
3.	User selects or updates their preferences.	
4.	User clicks Save.	
5.		System saves the preferences and updates recommendation engine settings.
6.		System displays a confirmation message.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
5.		System detects no changes and displays: No changes to save.

Identifier	UC-05	
Name	View Order History	
Summary	A logged-in user views their past orders placed through the PocketPlate system.	
Priority	Medium	
Actors	User	
Pre-condition(s)	User is logged in. User has placed at least one order previously.	
Post-condition(s)	User can view the details and statuses of all previous orders.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User navigates to Order History section.	
2.		System retrieves and displays a list of past orders with date, items, and total.
3.	User selects an order to view details.	
4.		System displays full details of the selected order.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
2.		System displays: You have no previous orders.

Identifier	UC-06
Name	Add Reviews
Summary	A user who has completed an order submits a review and rating for a restaurant.
Priority	Medium
Actors	User
Pre-condition(s)	User is logged in. User has at least one completed order from the restaurant.
Post-condition(s)	Review and rating are saved and displayed on the restaurant's page.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User navigates to a completed order and clicks Add Review.	
2.		System displays the review form with a rating scale and comment field.
3.	User selects a star rating and enters a comment.	
4.	User clicks Submit Review.	
5.		System validates and saves the review.
6.		System displays the review on the restaurant detail page.

Alternate Course of Action		
S#	**Actor Action**	**System Response**
5.		System displays an error: Please provide a rating before submitting.
6.	User selects a rating and resubmits.	
7.		(5) Continue from S# 5.

Identifier	UC-07	
Name	Enter Budget	
Summary	User enters a food spending budget for a given period to enable budget-based features.	
Priority	High	
Actors	User	
Pre-condition(s)	User is logged in.	
Post-condition(s)	Budget is set in the system and used for filtering and recommendations.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User navigates to the Budget section.	
2.		System displays the budget entry form.
3.	User enters the desired budget amount and period.	
4.	User clicks Set Budget.	
5.		System saves the budget and confirms the entry.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
5.		System displays an error: Please enter a valid positive amount.
6.	User corrects the amount.	
7.		(5) Continue from S# 5.

Identifier	UC-08	
Name	Apply Budget Filter	
Summary	User applies a budget filter to see only restaurants and meals within their set budget.	
Priority	High	
Actors	User	
Pre-condition(s)	User is logged in. User has a budget set (UC-07).	
Post-condition(s)	Displayed restaurants and meal options are filtered to match the user's budget.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User enables the Apply Budget Filter toggle.	
2.		System applies the budget filter to all restaurant and meal listings.
3.		System refreshes the display showing only budget-compliant options.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
3.		System displays: No meals found within your budget. Consider adjusting your budget.

Identifier	UC-09	
Name	Activate End-Of-Month Mode	
Summary	User activates end-of-month mode which applies stricter budget constraints as the month nears its end.	
Priority	Low	
Actors	User	
Pre-condition(s)	User is logged in. User has an active budget set.	
Post-condition(s)	System operates in end-of-month mode showing only the most budget-friendly options.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User navigates to the Budget section.	
2.	User clicks Activate End-Of-Month Mode.	
3.		System enables end-of-month mode and applies tighter budget constraints.
4.		System notifies the user that end-of-month mode is now active.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
3.		System prompts user to set a budget before activating the mode.
4.	User sets a budget (UC-07).	
5.		(3) Continue from S# 3.

Identifier	UC-10	
Name	Browse Restaurants	
Summary	User browses available restaurants listed on the PocketPlate platform.	
Priority	High	
Actors	User	
Pre-condition(s)	User is logged in. At least one restaurant is registered in the system.	
Post-condition(s)	User sees a list of available restaurants.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User navigates to the Browse Restaurants page.	
2.		System retrieves and displays a list of available restaurants.
3.	User scrolls through the restaurant listing.	
Alternate Course of Action		
S#	**Actor Action**	**System Response**
2.		System displays: No restaurants are currently available. Please check back later.

Identifier	UC-11	
Name	View Restaurant Detail	
Summary	User views detailed information about a specific restaurant including its menu, ratings, and location.	
Priority	High	
Actors	User	
Pre-condition(s)	User is logged in and has accessed the Browse Restaurants page.	
Post-condition(s)	User sees the full details and menu of the selected restaurant.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User selects a restaurant from the list.	
2.		System retrieves and displays restaurant details (menu, rating, location, hours).
3.	User browses the menu items.	
Alternate Course of Action		
S#	**Actor Action**	**System Response**
2.		System displays an error: Unable to load restaurant details. Please try again.

Identifier	UC-12	
Name	View Newly Opened Restaurants	
Summary	User views a list of restaurants that have recently been added to the platform.	
Priority	Low	
Actors	User	
Pre-condition(s)	User is logged in.	
Post-condition(s)	User is presented with a list of newly opened restaurants.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User selects Newly Opened from the restaurant section.	
2.		System filters and displays restaurants added within the last 30 days.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
2.		System displays: No newly opened restaurants at this time.

Identifier	UC-13	
Name	View Daily Updates Page	
Summary	User views the daily updates page showing current offers, vouchers, and card-based discounts.	
Priority	Medium	
Actors	User	
Pre-condition(s)	User is logged in.	
Post-condition(s)	User sees daily offers, vouchers, and discounts available for the day.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User navigates to Daily Updates.	
2.		System retrieves and displays today's offers, vouchers, and discounts.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
2.		System displays: No daily updates available at this time.

Identifier	UC-14	
Name	View Promotions	
Summary	User browses ongoing promotions offered by restaurants on the platform.	
Priority	Medium	
Actors	User	
Pre-condition(s)	User is logged in.	
Post-condition(s)	User can see and select promotions to apply to their orders.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User navigates to the Promotions section.	
2.		System retrieves and displays current promotions from all restaurants.
3.	User selects a promotion to view its details.	
4.		System displays the promotion details and applicable conditions.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
2.		System displays: No promotions are currently available.

Identifier	UC-15	
Name	View Daily Offers	
Summary	User views daily special offers provided by restaurants. This use case is included in View Daily Updates Page (UC-13).	
Priority	Medium	
Actors	User	
Pre-condition(s)	User is logged in and has navigated to the Daily Updates Page.	
Post-condition(s)	User can see all daily offers and their details.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User opens the Daily Updates Page (UC-13).	
2.	User selects the Daily Offers tab.	
3.		System retrieves and displays today's special offers.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
3.		System displays: No daily offers available today.

Identifier	UC-16	
Name	View Vouchers	
Summary	User views available vouchers that can be redeemed for discounts on orders.	
Priority	Medium	
Actors	User	
Pre-condition(s)	User is logged in.	
Post-condition(s)	User can see and save vouchers to use at checkout.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User opens the Daily Updates Page or Vouchers section.	
2.		System displays available vouchers with details and expiry dates.
3.	User selects a voucher to save or use.	
Alternate Course of Action		
S#	**Actor Action**	**System Response**
2.		System displays: No vouchers are currently available for you.

Identifier	UC-17	
Name	View Student-Exclusive Vouchers	
Summary	User with a verified student account views vouchers exclusively available to students.	
Priority	Low	
Actors	User (Student)	
Pre-condition(s)	User is logged in and has a verified student account.	
Post-condition(s)	User sees student-exclusive vouchers and can apply them to orders.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User navigates to the Vouchers section and selects Student Vouchers.	
2.		System verifies student status and retrieves student-exclusive vouchers.
3.		System displays the list of student-exclusive vouchers.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
2.		System displays: Student verification required. Please verify your student status to access these vouchers.

Identifier	UC-18	
Name	View Card Based Discounts	
Summary	User views discounts available based on their registered payment card type or bank.	
Priority	Medium	
Actors	User	
Pre-condition(s)	User is logged in and has a payment card registered.	
Post-condition(s)	User can see card-specific discounts applicable to their orders.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User navigates to the Card Based Discounts section.	
2.		System retrieves and displays discounts associated with the user's registered card.
3.	User reviews the applicable discounts.	
Alternate Course of Action		
S#	**Actor Action**	**System Response**
2.		System prompts user to add a payment card to view card-based discounts.

Identifier	UC-19	
Name	Enable/Set Location	
Summary	User enables location services and sets their current or preferred delivery location.	
Priority	High	
Actors	User	
Pre-condition(s)	User is logged in. Device location services are available.	
Post-condition(s)	User's location is set and used for nearby restaurant suggestions and delivery.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User navigates to location settings or is prompted to set location.	
2.	User clicks Enable Location or enters an address manually.	
3.		System requests location permission from the device.
4.		System detects and sets the user's current location.
5.		System updates restaurant listings based on the location.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
4.		System prompts user to enter location manually as permission was denied.
5.	User enters address manually.	
6.		(5) Continue from S# 5.

Identifier	UC-20	
Name	Get Optimised Meal Recommendation	
Summary	User receives AI-powered meal recommendations optimized based on budget, preferences, and order history.	
Priority	High	
Actors	User	
Pre-condition(s)	User is logged in. Preferences and budget are set.	
Post-condition(s)	User receives a list of optimized meal recommendations.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User navigates to Meal Recommendations.	
2.		System retrieves user preferences, budget, location, and order history.
3.		System runs the AI optimization algorithm.
4.		System displays a ranked list of recommended meals within budget.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
4.		System displays generic popular meals and prompts user to set preferences for personalised results.

Identifier	UC-21	
Name	Build Cross-Restaurant Combo	
Summary	User builds a custom meal combination by selecting items from multiple restaurants.	
Priority	High	
Actors	User	
Pre-condition(s)	User is logged in. Multiple restaurants are available.	
Post-condition(s)	A cross-restaurant combo cart is created ready for checkout.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User navigates to Build Cross-Restaurant Combo.	
2.		System displays available restaurants and menu items.
3.	User selects items from different restaurants.	
4.		System adds selected items to a combined cart.
5.	User reviews the combo and proceeds to checkout.	
Alternate Course of Action		
S#	**Actor Action**	**System Response**
4.		System warns: This combination exceeds your budget. Consider removing items.
5.	User adjusts selection.	
6.		(4) Continue from S# 4.

Identifier	UC-22	
Name	Get Group Recommendation & Bill Split	
Summary	User gets meal recommendations for a group and the system calculates how to split the bill among members.	
Priority	Medium	
Actors	User	
Pre-condition(s)	User is logged in. Group members and budget are specified.	
Post-condition(s)	System provides group meal recommendations and displays individual bill split amounts.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User selects Get Group Recommendation & Bill Split.	
2.	User adds group members (UC-23) and enters group budget (UC-24).	
3.		System generates optimized meal recommendations for the group.
4.		System calculates and displays the individual bill split.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
3.		System warns: Budget is insufficient for all members. Please increase the budget or reduce group size.

Identifier	UC-23	
Name	Add Members	
Summary	User adds members to a group ordering session for group recommendations and bill splitting.	
Priority	Medium	
Actors	User	
Pre-condition(s)	User is logged in and is initiating a group recommendation.	
Post-condition(s)	Members are added to the group session.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User selects Add Members in the group recommendation screen.	
2.		System displays a search/input field to add members.
3.	User enters member names or searches by username.	
4.		System adds the selected members to the group.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
4.		System displays: User not found. Please check the username and try again.

Identifier	UC-24	
Name	Enter Group Budget	
Summary	User enters a total budget for a group dining or ordering session.	
Priority	Medium	
Actors	User	
Pre-condition(s)	User is logged in and is in an active group recommendation session.	
Post-condition(s)	Group budget is set and used for generating group meal recommendations.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User enters the total group budget in the provided field.	
2.	User confirms the budget.	
3.		System saves the group budget and applies it to the recommendation engine.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
3.		System displays: Please enter a valid positive number for the group budget.
4.	User corrects and re-enters the amount.	
5.		(3) Continue from S# 3.

Identifier	UC-25	
Name	Customise Individual Share	
Summary	User customizes their individual portion within a group order.	
Priority	Low	
Actors	User	
Pre-condition(s)	User is part of an active group order session.	
Post-condition(s)	User's individual selection is updated within the group order.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User selects Customise My Share in the group order.	
2.		System displays the user's current item selection.
3.	User modifies item selections (add/remove items).	
4.		System updates the user's share and recalculates the bill split.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
4.		System warns: Your selection exceeds your individual share. Please adjust items.

Identifier	UC-26	
Name	Select Payment Card	
Summary	User selects a saved payment card to use for completing a purchase.	
Priority	High	
Actors	User	
Pre-condition(s)	User is at the payment screen and has at least one saved payment card.	
Post-condition(s)	Selected card is marked as the active payment method for the transaction.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.		System displays saved payment cards during checkout.
2.	User selects the desired payment card.	
3.		System sets the selected card as the active payment method.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
1.		System prompts user to add a new payment card as no cards are saved.
2.	User enters new card details.	
3.		System saves the card and sets it as the payment method.

Identifier	UC-27	
Name	Add Items to Cart	
Summary	User adds desired food items to their cart from a restaurant's menu.	
Priority	High	
Actors	User	
Pre-condition(s)	User is logged in and has selected a restaurant.	
Post-condition(s)	Selected items are added to the user's cart.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User browses a restaurant's menu.	
2.	User selects a menu item and clicks Add to Cart.	
3.		System adds the item to the cart and updates the cart total.
4.		System displays a confirmation notification.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
3.		System displays: Sorry, this item is currently unavailable.

Identifier	UC-28	
Name	Manage Multi-Restaurant Cart	
Summary	User manages a cart containing items from multiple restaurants simultaneously.	
Priority	High	
Actors	User	
Pre-condition(s)	User is logged in and has items from multiple restaurants in the cart.	
Post-condition(s)	Cart is updated with correct items, quantities, and totals per restaurant.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User opens the cart.	
2.		System displays items grouped by restaurant with subtotals.
3.	User modifies item quantities or removes items.	
4.		System updates quantities and recalculates totals.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
4.		System clears the cart and displays: Your cart is empty.

Identifier	UC-29	
Name	Place Order	
Summary	User finalizes and places their order from the cart through the PocketPlate system.	
Priority	High	
Actors	User, External Restaurant System	
Pre-condition(s)	User is logged in. Cart has at least one item. Delivery address is set.	
Post-condition(s)	Order is placed and confirmed. External Restaurant System receives the order.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User reviews the cart and clicks Place Order.	
2.		System displays order summary with items, totals, and delivery info.
3.	User confirms the order.	
4.		System processes the order and forwards it to the External Restaurant System.
5.		System confirms the order and displays order tracking details.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
5.		System displays: Unable to reach the restaurant. Please try again later.

Identifier	UC-30
Name	Make Payment
Summary	User completes payment for their order using a selected payment card and optional vouchers.
Priority	High
Actors	User
Pre-condition(s)	User has confirmed an order (UC-29). User has a registered payment method.
Post-condition(s)	Payment is processed and order status is updated to Paid.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User proceeds to payment from the order summary.	
2.		System displays payment options and saved cards.
3.	User selects a payment card (UC-26) and applies any vouchers (UC-31).	
4.	User confirms payment.	
5.		System processes the payment via the payment gateway.
6.		System confirms payment and updates order status to Paid.

Alternate Course of Action		
S#	**Actor Action**	**System Response**
6.		System displays: Payment declined. Please check your card details or try another card.
7.	User selects another card or retries.	
8.		(5) Continue from S# 5.

Identifier	UC-31	
Name	Apply Vouchers	
Summary	User applies available voucher codes at checkout to receive a discount on their order.	
Priority	Medium	
Actors	User	
Pre-condition(s)	User is at the payment screen. User has at least one valid voucher.	
Post-condition(s)	Discount is applied and the order total is reduced accordingly.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	User selects Apply Voucher at checkout.	
2.		System displays the user's available vouchers.
3.	User selects a voucher to apply.	
4.		System validates the voucher and applies the discount to the order total.
5.		System displays the updated total with the discount applied.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
4.		System displays: This voucher is invalid or has expired.
5.	User selects a different voucher or proceeds without one.	

Identifier	UC-32	
Name	Manage Users	
Summary	Admin manages all user accounts including viewing, editing, suspending, or deleting accounts.	
Priority	High	
Actors	Admin	
Pre-condition(s)	Admin is logged in with appropriate privileges.	
Post-condition(s)	User account changes are applied and saved in the system.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	Admin navigates to Manage Users.	
2.		System displays a list of all registered users.
3.	Admin selects a user to view or edit.	
4.		System displays user details and action options (edit, suspend, delete).
5.	Admin performs the desired action.	
6.		System applies and confirms the changes.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
6.		System warns: This user has active orders. Confirm deletion?
7.	Admin confirms deletion.	
8.		System deletes the user and cancels active orders.

Identifier	UC-33	
Name	Manage Restaurants	
Summary	Admin manages restaurant listings including adding, editing, approving, or removing restaurants.	
Priority	High	
Actors	Admin	
Pre-condition(s)	Admin is logged in.	
Post-condition(s)	Restaurant data is updated in the system and reflected on the platform.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	Admin navigates to Manage Restaurants.	
2.		System displays all registered restaurants.
3.	Admin selects a restaurant to manage.	
4.		System displays restaurant details and available actions.
5.	Admin edits, approves, or removes the restaurant.	
6.		System saves changes and updates the restaurant listing.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
6.		System warns: This restaurant has active orders. Confirm removal?
7.	Admin confirms.	
8.		System removes the restaurant and notifies affected users.

Identifier	UC-34	
Name	Manage Restaurant Integration	
Summary	Admin manages integration settings for connecting external restaurant systems to PocketPlate.	
Priority	High	
Actors	Admin	
Pre-condition(s)	Admin is logged in. External restaurant system details are available.	
Post-condition(s)	Integration settings are saved and the external restaurant system is connected.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	Admin navigates to Restaurant Integration.	
2.		System displays current integration configurations.
3.	Admin adds or updates the API credentials and settings for a restaurant.	
4.		System validates the credentials by testing the connection.
5.		System saves the configuration and marks the integration as active.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
5.		System displays: Connection failed. Please check the API credentials and try again.
6.	Admin corrects the credentials.	
7.		(4) Continue from S# 4.

Identifier	UC-35	
Name	Update Offers and Promotions Data	
Summary	Admin updates the daily offers, promotions, and discount data displayed to users.	
Priority	Medium	
Actors	Admin	
Pre-condition(s)	Admin is logged in.	
Post-condition(s)	Updated offers and promotions are saved and visible to users.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	Admin navigates to Update Offers & Promotions.	
2.		System displays existing offers and promotions with edit options.
3.	Admin adds, edits, or removes offers and promotions.	
4.	Admin clicks Save.	
5.		System saves and publishes the updated offers and promotions.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
5.		System highlights invalid fields and prompts admin to correct them.
6.	Admin corrects the data.	
7.		(5) Continue from S# 5.

Identifier	UC-36	
Name	Monitor System Data	
Summary	Admin monitors real-time system data including server health, active sessions, and performance metrics.	
Priority	Medium	
Actors	Admin	
Pre-condition(s)	Admin is logged in.	
Post-condition(s)	Admin can view real-time system metrics and take action if needed.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1.	Admin navigates to Monitor System Data.	
2.		System displays real-time metrics (server status, active users, error logs).
3.	Admin reviews the data and monitors for anomalies.	
Alternate Course of Action		
S#	**Actor Action**	**System Response**
3.		System highlights an alert (e.g., high error rate or downtime).
4.	Admin investigates and takes corrective action.	